Session 12: Inside the GPU

Teaching guide for `12_gpu_architecture_scheduling/slides.html`: compute architecture and scheduling (based on chapter 4 of *Programming Massively Parallel Processors*).

HOW TO USE THIS GUIDE

- One section per slide, in deck order: the goal of the slide, what to say, how to drive its diagram, questions to ask (with answers), and pitfalls.
- Open the deck in a browser (serve the repository with `python3 -m http.server 8767` and open `http://localhost:8767/12_gpu_architecture_scheduling/slides.html`). Diagrams start playing when a slide opens; Step and Reset let you control them while you talk.
- Suggested time: about 143 minutes including the exercises; the timing per slide is a guide, not a target.
- All numbers on the slides (block scheduling, barrier waiting time, divergent warps, latency utilisation, occupancy, exercise answers) were computed by script and checked against the animations.

PLAN OF THE SESSION

Part	Slides	Minutes
Architecture and block scheduling	1–6	20
Barriers and transparent scalability	7–12	24
Warps and SIMD hardware	13–18	23
Control divergence	19–23	20
Latency tolerance and occupancy	24–30	26
Summary, exercises, solutions, self-study	31–37	30

BEFORE CLASS

- Open slides 22, 23 and 27 once: they are calculators, and students like to propose their own numbers.
- Have chapter 4 at hand for figures 4.1–4.10; every one of them is redrawn and animated in the deck.

SLIDE 1 Title

≈ 1 min

Goal. Set the question for the session: where do the threads of a kernel actually run?

SAY

- Recall session 11: we chose `<<<grid, block>>>` without asking where those threads physically go.
- Today's promise: by the end, students can predict why one launch configuration is faster than another without running it.

DRIVE THE DIAGRAM

- Mention the controls once: arrows or space to move, Play/Step/Reset on every diagram, `p` to print.

SLIDE 2 From the programming model to the machine

≈ 3 min

Goal. Connect the hardware topics to performance questions students already have.

SAY

- CPU: few powerful cores, big caches and branch prediction, all to make one thread finish fast (latency).
- GPU: many simple processors and a huge register file, to finish as many threads per second as possible (throughput).
- Walk the table: each hardware topic answers one practical question. Tell them every later slide maps to a row.

ASK

> **Q.** Why might 32 threads per block be slower than 256 even if the total number of threads is the same?
> **A.** Keep the question open; it is answered on the occupancy slides (block slots run out at 32 threads per block).

SLIDE 3 Architecture of a modern GPU (Fig. 4.1)

≈ 5 min

Goal. Name the levels: GPU, GPC, SM, streaming processor, on-chip memory, global memory.

SAY

- Let the animation highlight each level; pause on the SM step and point at the three parts: control, processors, memory.
- H100 numbers: 132 SMs, 128 streaming processors each (16,896 in total), arranged in 8 GPCs.
- Two memories: small, fast on-chip memory per SM (chapter 5) and large, slow-ish off-chip global memory reached through many controllers (chapter 6).

DRIVE THE DIAGRAM

- Play runs through the six steps (3 s each). Use Step if you want to talk longer about one level.

ASK

> **Q.** Why does the global memory have several controllers instead of one?
> **A.** Bandwidth: many controllers serve requests in parallel, so thousands of threads can be fed at once.

WATCH OUT

- Do not call streaming processors “cores”: students then compare them one-to-one with CPU cores, which is misleading.

SLIDE 4 Vocabulary, with one real GPU as a yardstick

≈ 3 min

Goal. Fix the vocabulary and give a sense of scale.

SAY

- Read the table top to bottom; stress the last row: threads, blocks and registers per SM are shared by all resident blocks.
- These three limits come back in the occupancy part, so ask students to remember 2048 / 32 / 65,536.

ASK

> **Q.** How many threads can an H100 hold at the same time?
> **A.** 132 SMs × 2048 threads = 270,336 threads resident at once.

SLIDE 5 Thread block scheduling (Fig. 4.2)

≈ 5 min

Goal. Blocks go to SMs whole, a few at a time, and freed slots are refilled.

SAY

- Ten blocks with different lengths (shown as “3t” = 3 time units) and two SMs that hold 3 blocks each.
- At time 0 six blocks start; the rest wait in the grid. Each time a block finishes, the next waiting block takes its slot.
- Point out that a block never moves between SMs and never runs partially: all its threads start together on one SM.

DRIVE THE DIAGRAM

- Let it play once (1.5 s per step), then Reset and Step through times 1 and 2 while asking who starts next.

ASK

> **Q.** At time 2, which blocks start and where?
> **A.** B7 and B8, both on SM 1, in the slots freed by B1 and B3.

> **Q.** Why is it useful that blocks may finish in any order?
> **A.** The scheduler can keep every slot busy without waiting for a particular block.

SLIDE 6 What block-by-block assignment guarantees

≈ 3 min

Goal. Explain what the whole-block rule makes possible: barriers and shared memory inside a block.

SAY

- Same SM + same time = threads of a block can wait for each other and share fast on-chip memory.
- Different blocks: no guarantee of running together, so no cooperation between them.
- Hopper's clusters add a level: blocks of a cluster are co-scheduled on one GPC and can sync and share memory. Mention it, do not dwell.

SLIDE 7 Barrier synchronization (Fig. 4.3)

≈ 5 min

Goal. Build the timing intuition of a barrier: early arrivals wait for the last one.

SAY

- Use the shopping-mall analogy in your own words: friends shop separately, then everyone waits at the car until the last one returns.
- Solid bar = working, hatched = waiting at `__syncthreads()`, green = after the barrier. The red line appears when the last thread (thread 4) arrives.
- The caption computes the waiting share: 10.6 of 31.2 thread-µs, about 34 %. This is exactly exercise 12.3.

DRIVE THE DIAGRAM

- The time cursor moves in 0.2 µs steps; Play takes about 6 s. Pause right after the release to read the caption.

ASK

> **Q.** What determines when everybody is released?
> **A.** The slowest thread: here thread 4 at 5.2 µs.

> **Q.** How could the waiting share be reduced?
> **A.** Balance the work so all threads reach the barrier at about the same time.

SLIDE 8 A barrier every thread must reach (Fig. 4.4)

≈ 5 min

Goal. Show why `__syncthreads()` inside an if/else is wrong.

SAY

- Start in Wrong mode: even threads stop at barrier A, odd at barrier B. Each call site is a separate barrier and waits for all 8 threads.
- Neither ever completes: undefined behaviour, typically a hang. Then switch to Right: one barrier after the branch, 8 of 8, released.
- Rule to write on the board: a `__syncthreads()` must be reached by all threads of the block or by none.

DRIVE THE DIAGRAM

- Use the Wrong/Right buttons; each mode has 4 steps. The code on the left switches with the mode.

ASK

> **Q.** Is `if (blockIdx.x == 0) __syncthreads();` legal?
> **A.** Yes: the condition is the same for every thread of a block, so either all of them reach it or none does.

WATCH OUT

- Students often think the two calls are “the same barrier” because the function is the same. Insist: the barrier is the call site.

SLIDE 9 Barrier scope and why it is limited to a block

≈ 3 min

Goal. Introduce scope (block, cluster, grid) and the resource-based reason for limiting it.

SAY

- A barrier needs every thread in its scope to be able to arrive. CUDA guarantees that inside a block by giving a block all its resources before it starts.
- Grid-wide barriers exist (Cooperative Groups) but only work if the whole grid fits on the GPU at once, and they are expensive.
- Bridge: because blocks never wait for each other, the hardware is free to run them in any order, which is the next slide.
- The note at the bottom: `__syncthreads()` is an intrinsic (the book's sidebar). The double underscore marks it; the compiler emits one special instruction, not a function call.

ASK

> **Q.** Why would a grid-wide barrier deadlock if the grid is larger than what fits on the GPU?
> **A.** Blocks that are waiting at the barrier keep their SM slots; the blocks not yet scheduled can never start, so they never arrive.

SLIDE 10 Code: a barrier in a real kernel

≈ 3 min

Goal. Show the barrier rule in working code before moving on.

SAY

- Read the three numbered steps: write my slot, wait, read another thread's slot.
- Ask what goes wrong without line 2 (a thread reads a slot that is not written yet: a race whose result changes between runs).
- Point out that the barrier is outside any branch, and that each block works alone, which is what lets the grid scale (next slide).

ASK

> **Q.** Would the kernel still be correct if thread t read s[t] instead of s[BLOCK - 1 - t]?
> **A.** Yes, even without the barrier: each thread would only read the slot it wrote itself. The barrier is needed because data moves between threads.

SLIDE 11 Transparent scalability (Fig. 4.5)

≈ 4 min

Goal. The same code runs on a small and a large GPU, faster on the large one, without changes.

SAY

- Device A runs 2 blocks at a time (4 waves); device B runs 4 at a time (2 waves).
- Point at the centre grid turning green: B finishes at step 2, A at step 4.
- This works only because blocks are independent. Connect to the market: phones, laptops and data-centre GPUs run the same CUDA program.

DRIVE THE DIAGRAM

- Six steps at 1.8 s; stop at step 2 and ask where device A is.

ASK

> **Q.** What would break transparent scalability?
> **A.** Blocks waiting for each other (for example spinning on a flag set by another block): on a small GPU the other block might never be scheduled.

SLIDE 12 Waves and the tail effect

≈ 4 min

Goal. Quantify waves: blocks in the grid divided by blocks that fit at once.

SAY

- Preset 660 / 264: 2.5 waves; the last wave uses half the GPU (tail effect).
- Preset 528 / 264: exactly 2 waves, no tail.
- Many waves help when block work is uneven, because the hardware rebalances; few, even waves are fine when work is balanced (thread coarsening, chapter 6).

DRIVE THE DIAGRAM

- Type numbers or use the presets; the bars and the utilisation update live.

ASK

> **Q.** A GPU fits 264 blocks at once. How many blocks would you choose for a balanced kernel?
> **A.** A multiple of 264, for example 528 or 792, so every wave is full.

SLIDE 13 Warps (Fig. 4.6)

≈ 4 min

Goal. Blocks are cut into warps of 32 consecutive threads; warps are what the SM schedules.

SAY

- Default: 3 blocks of 256 threads = 8 warps each = 24 warps in the SM (the book's example).
- Preset 48 × 3: each block needs 2 warps, the second padded with 16 inactive lanes (hatched).
- Formula on the board: warps per block = ceil(threads per block / 32); warp w holds threads 32w … 32w+31.

DRIVE THE DIAGRAM

- The highlight walks through the warps of block 0; type any block size to rebuild.

ASK

> **Q.** Why are block sizes usually multiples of 32?
> **A.** Otherwise the last warp of every block carries idle lanes that still occupy hardware.

SLIDE 14 2D/3D blocks become one line (Fig. 4.7)

≈ 4 min

Goal. Show how multidimensional blocks are linearised (row-major) before forming warps.

SAY

- Labels are (threadIdx.y, threadIdx.x). Each step moves one row into the linear order: all y = 0 first, then y = 1, and so on.
- 4 × 4: 16 threads, half a warp, padded with 16 inactive lanes. Switch to 8 × 8: 64 threads, warp 0 = rows 0–3, warp 1 = rows 4–7.
- Link to session 11: this is the same row-major rule as flattening a 2D array.

DRIVE THE DIAGRAM

- Use the 4×4 / 8×8 buttons; each row moves with an animation.

ASK

> **Q.** In a 16 × 16 block, which threads form warp 3?
> **A.** Linear positions 96–127: rows y = 6 and y = 7, all x.

> **Q.** For a 3D block, which index varies slowest?
> **A.** threadIdx.z: all z = 0 threads come first.

SLIDE 15 Code: which warp and which lane am I?

≈ 3 min

Goal. Turn the linearization rule into the two lines every warp-aware kernel uses.

SAY

- warp = t / warpSize and lane = t % warpSize, with t the linear index of the thread inside its block.
- Walk the table: in a 16-wide block, every two rows form one warp; thread (5, 3) is t = 53, warp 1, lane 21.
- Use warpSize rather than a literal 32.

ASK

> **Q.** In a 32 × 8 block, which threads form warp 3?
> **A.** t = 96…127: the whole row threadIdx.y = 3.

SLIDE 16 Processing blocks and SIMD (Fig. 4.8)

≈ 4 min

Goal. One instruction fetch serves many lanes: SIMD hardware, programmed as SIMT.

SAY

- Toy SM: two processing blocks of 8 lanes with toy 8-thread warps. Real H100: four processing blocks of 32 lanes, 32-thread warps.
- Each step: one processing block fetches one instruction of one warp and sends it to all its lanes; each lane has different register values, so different results.
- SIMT: we write a scalar thread; the hardware groups threads into warps and runs them in SIMD.

DRIVE THE DIAGRAM

- Eight steps alternate between the two processing blocks and the four warps.

ASK

> **Q.** What is shared by the lanes and what is private?
> **A.** Shared: instruction fetch/dispatch (control). Private: each lane's registers (data).

SLIDE 17 Von Neumann model to SIMD (sidebar)

≈ 5 min

Goal. Explain the motivation for warps from first principles.

SAY

- One processor mode: walk the fetch–decode–execute cycle (PC → memory → IR → ALU, PC advances). This is the stored-program model.
- GPU mode: the same control unit drives four processing units, each with its own registers (r2, r3 differ, so r1 differs).
- The payoff: control hardware is paid once, so more transistors go to arithmetic. The price: threads in a warp must follow the same instructions, hence divergence next.

DRIVE THE DIAGRAM

- Toggle “One processor” / “GPU processing block”; five steps each.

ASK

> **Q.** Name Flynn's four classes and place a CPU core and a GPU processing block.
> **A.** SISD, SIMD, MISD, MIMD. A scalar CPU core is SISD (ignoring its vector units); a GPU processing block is SIMD; a multicore CPU, or a GPU as a whole with many SMs, is MIMD.

SLIDE 18 SIMD, SIMT and Flynn's taxonomy

≈ 3 min

Goal. Name the classes and explain what SIMT adds on top of SIMD.

SAY

- Flynn's four classes: SISD, SIMD, MISD, MIMD. A GPU processing block is SIMD; the GPU as a whole (many SMs with their own instruction streams) is MIMD.
- SIMT: the programmer writes scalar threads; the hardware forms warps. On a CPU, SIMD requires explicit vector instructions or intrinsics.
- Warp size is implementation-defined (32 so far): read `warpSize`. Warp-level primitives appear in chapters 10–12.
- End with the price of SIMD: divergence, which is the next slide.

ASK

> **Q.** Where does MISD appear in practice?
> **A.** Rarely; some fault-tolerant designs run several instruction streams on the same data and compare results.

SLIDE 19 Control divergence in an if/else (Fig. 4.9)

≈ 5 min

Goal. Different paths inside a warp cost extra passes.

SAY

- threadIdx.x < 24: pass A on 24 lanes, pass B on 8 lanes, then C on all 32. SIMD efficiency over the if/else: 50 %.
- threadIdx.x % 2 == 0: the worst pattern, every warp splits 16/16.
- threadIdx.x > 2 (the book's text example): only the first warp of a block diverges; other warps are all-true.
- blockIdx.x == 0: the same for the whole warp, so no divergence, one pass.

DRIVE THE DIAGRAM

- Use the four condition buttons; each runs the four-step sequence.

ASK

> **Q.** Does `if (threadIdx.x / 32 == 0)` diverge?
> **A.** No: it is constant inside each warp (it selects whole warps).

> **Q.** What is the cost of divergence?
> **A.** Extra passes, plus the lanes that sit idle during each pass.

SLIDE 20 Divergence in loops; independent thread scheduling (Fig. 4.10)

≈ 4 min

Goal. Loops with per-thread trip counts keep the warp busy until the longest thread ends.

SAY

- The lanes of Fig. 4.10 (threads 0, 1, 2, 4, 5, 6, 7, 31) have trip counts 8, 6, 7, 4, 5, 6, 8, 7: the warp iterates 8 times; useful work 51 of 64 lane-slots (≈ 80 %).
- Pascal and older ran divergent paths one after the other; from Volta on they may be interleaved (independent thread scheduling).
- Consequence: never assume a warp is back in lockstep after a branch; use `__syncwarp()` when a phase must complete warp-wide.

DRIVE THE DIAGRAM

- Play shows one iteration per step; the caption marks divergent iterations.

ASK

> **Q.** Which iterations are free of divergence here?
> **A.** Iterations i = 0 to 3: every lane has n ≥ 4.

SLIDE 21 Code: divergent and warp-uniform branches

≈ 4 min

Goal. Compare the same work written in ways that do and do not diverge.

SAY

- A: the condition alternates lane by lane, so every warp runs both paths.
- B: the condition is constant inside a warp (it depends on t / warpSize), so each warp runs one path. Same work, half the passes.
- C: the boundary check diverges in at most one warp. D: for tiny branches, compute both values and select; compilers often do this themselves (predication).
- Remind them: never put __syncthreads() after an early return taken by only some threads.

ASK

> **Q.** When can you not use the B form?
> **A.** When the kind of work is decided by the data (for example the value of x[i]); then divergence depends on the input.

SLIDE 22 The boundary check in 1D

≈ 3 min

Goal. Boundary checks are the most common source of divergence, and their cost shrinks with size.

SAY

- 1003 / 64: 16 blocks, 1024 threads, 32 warps; only the warp holding i = 1003 diverges (≈ 3 %).
- 10000 / 256: 320 warps launched, one divergent, 7 entirely idle (idle warps are not divergent).
- The share of divergent warps falls as n grows; for realistic sizes it is negligible.

DRIVE THE DIAGRAM

- Change n and the block size (multiples of 32) or use the presets.

ASK

> **Q.** With n = 3000 and 256 threads per block, how many warps diverge?
> **A.** One (warp 93, threads 2976–3007): exercise 12.2.

SLIDE 23 Boundary checks in 2D

≈ 4 min

Goal. Count divergent warps in a 2D launch.

SAY

- With 16 × 16 blocks each warp is two rows of 16 threads.
- 76 × 62 (session 11's picture): 20 blocks, 160 warps, 31 divergent: the right-hand block column (24) plus 7 in the corner block. The bottom block row never diverges: its warps are fully in or fully out.
- 200 × 150: 1040 warps, **75** divergent (7.2 %). Note for the instructor: the book's text estimates 80 by counting all 8 warps of the corner block; the bottom 5 warps of that block are entirely outside the picture, so the exact count is 75.
- The larger the picture, the smaller the share (1000 × 1000: 500 of 31,752, 1.6 %).

DRIVE THE DIAGRAM

- Presets redraw the map; orange cells are divergent warps, grey are warps entirely outside. The output lists the book's four regions: 1 inside, 2 right column, 3 bottom row, 4 corner.
- Preset 76 × 63 answers the book's “reader should verify” remark: with an odd height the bottom-row warps (region 3) also diverge (36 divergent instead of 31).

ASK

> **Q.** Why does the bottom row of blocks not diverge when the height is 62?
> **A.** Rows 48–61 are inside and 62–63 outside; each warp covers two whole rows, so it is either fully inside or fully outside.

SLIDE 24 Warp scheduling and latency tolerance

≈ 5 min

Goal. Many resident warps let the SM hide memory latency.

SAY

- Each warp runs C C L C C L C C; a load (L) makes the warp wait 5 cycles. One execution unit picks a ready warp each cycle.
- 1 warp: busy 8 of 18 cycles (44 %). 2 warps: 67 %. 4 warps: 89 %. 6 warps: 100 %: some warp is always ready.
- Use the post-office analogy in your own words: the clerk serves the next customer while someone fills in a form.
- This is why an H100 SM holds up to 2048 threads while it has only 128 processors: 16× oversubscription.

DRIVE THE DIAGRAM

- Choose 1, 2, 4 or 6 warps; Play reveals one cycle every 0.33 s. Pause on an idle (pink) cycle and ask why.

ASK

> **Q.** Why do GPUs devote less area to caches and branch prediction than CPUs?
> **A.** They hide latency by switching warps instead, and spend the area on arithmetic units and memory channels.

SLIDE 25 Zero-overhead scheduling (sidebar)

≈ 3 min

Goal. Explain why switching warps is free while a CPU context switch is not.

SAY

- CPU: one register set per core; switching threads means saving A's registers to memory and loading B's.
- GPU: every resident warp keeps its registers in the SM's large register file, so the scheduler just picks another warp.
- Connect forward: that register file is a shared resource, which is exactly what limits occupancy.

SLIDE 26 How an SM's resources are partitioned

≈ 3 min

Goal. Introduce the four SM resources, dynamic partitioning and the definition of occupancy before the calculator.

SAY

- Resources: thread slots, block slots, registers, shared memory. Each resident block takes its share.
- Dynamic versus fixed partitioning: dynamic lets the SM run many small blocks or a few large ones; fixed would waste slots or reject large blocks.
- Read the table: block sizes 1024 to 64 all reach 2048 threads; 32 runs out of block slots. Registers: at most 32 per thread for full occupancy on an H100.

ASK

> **Q.** A block of 128 threads uses 48 registers per thread. Does it reach full occupancy?
> **A.** No: 65,536 / (128 × 48) = 10 blocks = 1280 threads, 62.5 %, limited by registers.

SLIDE 27 Resource partitioning and occupancy

≈ 6 min

Goal. Compute occupancy from threads, block slots and registers.

SAY

- Occupancy = resident warps / maximum warps (64 on an H100 SM).
- Resident blocks = min(floor(2048 / threads per block), 32, floor(65,536 / (threads per block × registers per thread))).
- Walk the presets: 1024 (100 %), 32 threads (50 %, block slots), 768 (75 %, 2048 not a multiple), 64 registers (50 %), 512 with 31 vs 33 registers (100 % vs 75 %).

DRIVE THE DIAGRAM

- Each preset resets the SM limits to H100 values; the warp-slot map shows one colour band per block.

ASK

> **Q.** What is the smallest block size that can reach 100 % occupancy?
> **A.** 64 threads: 32 blocks × 64 = 2048.

> **Q.** Two more local variables push a kernel from 31 to 33 registers. What happens?
> **A.** 4 blocks need 67,584 registers > 65,536, so only 3 fit: 75 % (a performance cliff).

WATCH OUT

- The book's model ignores register allocation granularity; real GPUs round registers up per warp. Mention it only if asked.

SLIDE 28 Occupancy in numbers

≈ 3 min

Goal. Consolidate the four ways to lose occupancy.

SAY

- Let students predict each row before you reveal the explanation column (cover it with a sheet or ask aloud).
- Explain register spilling: fewer registers per thread, more occupancy, but slower memory accesses per thread. Higher occupancy is not automatically faster.

SLIDE 29 Asking the GPU what it has

≈ 3 min

Goal. Show the device query API and the fields that matter for launch configuration.

SAY

- `cudaGetDeviceCount` counts devices (integrated GPUs too); `cudaGetDeviceProperties` fills a `cudaDeviceProp`.
- Fields tied to today: SM count, threads per block, threads per SM, registers, warp size, compute capability.
- `cudaOccupancyMaxActiveBlocksPerMultiprocessor` and the Nsight Compute calculator do the slide-27 computation for a real kernel (code on slide 30).

ASK

> **Q.** Why query at run time instead of hard-coding 1024 threads per block?
> **A.** Devices differ, and future devices may allow more; queried limits keep the program correct and fast everywhere.

SLIDE 30 Code: let the runtime compute occupancy

≈ 3 min

Goal. Show how occupancy is obtained for a real compiled kernel.

SAY

- cudaOccupancyMaxActiveBlocksPerMultiprocessor knows the kernel's real register and shared-memory use, which the calculator only assumes.
- cudaOccupancyMaxPotentialBlockSize suggests a block size; minGrid is the smallest grid that fills every SM.
- Mention nvcc -Xptxas -v to see registers per thread, and that higher occupancy is a means: always time the kernel.

SLIDE 31 Summary

≈ 2 min

Goal. Recap the five ideas.

SAY

- Ask students to say each bullet in one sentence before you show it, if time allows.

SLIDE 32 Exercise 12.1

≈ 8 min

Goal. Practise counting warps, divergence and SIMD efficiency on one kernel.

SAY

- Give 5–6 minutes. Hint if needed: work per block first (4 warps of 32), then multiply by 16 blocks.
- Line 4: warp 2 (threads 64–95) has no thread that passes; it is inactive, not divergent.

ASK

> **Q.** Answers
> **A.** See slide 34: 4 warps per block, 64 in the grid; line 4: 48 active, 32 divergent, efficiencies 100/50/0/50 %; line 6: 64 active, 64 divergent, 25 %; loop: 1 iteration without divergence, 3 with.

SLIDE 33 Exercises 12.2–12.9

≈ 10 min

Goal. Short exercises on divergence, barriers, warp size and occupancy.

SAY

- Pairs work well: one student computes, the other checks with the calculators on slides 22, 23 and 27.
- 12.4 is conceptual; make sure the answer mentions independent thread scheduling, not just “it might not work”.

SLIDE 34 Solutions 12.1

≈ 3 min

Goal. Go through the table; ask a student to explain each row before revealing it.

SAY

- Emphasise the distinction between an inactive warp (no thread passes) and a divergent one (some do, some do not).

SLIDE 35 Solutions 12.2–12.5

≈ 3 min

Goal. Correct and discuss.

SAY

- 12.3: the 22.9 % is the same computation as the barrier animation on slide 7.
- 12.5: point out that 384 is not a power of two but still a multiple of 32; it loses because 2048 / 384 is not an integer.

SLIDE 36 Solutions 12.6–12.9

≈ 3 min

Goal. Correct and discuss.

SAY

- 12.7(c): registers, not threads, are the limit; this is the slide-27 calculator with 128 threads and 40 registers.
- 12.8: show the 100 × 70 preset on slide 23 to confirm 35 divergent warps.
- 12.9: a launch that violates a limit fails; in real code always check `cudaGetLastError()` after a launch.

SLIDE 37 Self-study

≈ 1 min

Goal. Point to chapters 5 and 6 and the tools.

SAY

- Suggest running the device query on a lab machine and filling a table of SMs, threads per SM and compute capability.